

Spark/Flink [서정] @2월 마지막주 차

재영

@ 2월 28일까지 완료하기

@ 2월 22일 주차 (이내)

@ 아이디어 굳 / 구체화 필요

Ideation을 위해 확인한 문서

Spark

<https://spark.apache.org/docs/latest/index.html>

<https://spark.apache.org/docs/latest/cluster-overview.html>

<https://spark.apache.org/docs/latest/quick-start.html>

<https://spark.apache.org/docs/latest/job-scheduling.html>

<https://spark.apache.org/docs/latest/tuning.html>

Flink

<https://flink.apache.org/what-is-flink/flink-architecture/>

<https://flink.apache.org/what-is-flink/flink-applications/>

<https://flink.apache.org/what-is-flink/flink-operations/>

<https://flink.apache.org/what-is-flink/use-cases/>

Question

@ Spark vs Flink (차이점/작동방식)

@ latency vs throughput ?

@ cluster를 돌릴때 어떻게 작동하는지

@ 왜 필요한지 / 어떤 문제점에서 시작하였는지

@ 설정시 무엇을 만져서 성능을 향상 시키는지

@ 최근에는 무엇을 어떤 방식으로 활용하는지

1. 왜 Spark / Flink가 필요한가

1.1 데이터 처리 패러다임의 변화

1.2 현대 데이터 시스템의 특징

- ① 데이터 규모 증가
- ② 데이터는 지속적으로 생성된다
- ③ 실시간 요구 증가
- ④ 장애 필연적

1.3 핵심 문제

1.4 필요한 시스템 특성

- 수평 확장 (Horizontal Scaling)
- 병렬 처리 (Parallel Processing)
- 장애 복구 (Fault Tolerance)
- 분산 상태 관리 (Distributed State)

결론

2. 데이터 파이프라인에서의 위치

각 계층의 역할

Source Layer

Ingestion Layer

Processing Layer

역할: 데이터를 계산하고 변환하는 계층

Storage Layer

Serving Layer

3. Spark 설계 철학

Spark의 핵심 질문

Spark의 기본 모델

DAG (Directed Acyclic Graph)

Spark Cluster 구조

Driver

Executors

Spark 데이터 모델

RDD (Resilient Distributed Dataset)

Spark의 Fault Tolerance

4. Spark Streaming (Micro-batch)

Micro-batch 구조

장점

단점

5. Flink 설계 철학

Streaming-first 모델

6. Flink 실행 모델

Event Time 개념

7. Flink State + Checkpoint

Checkpoint

Exactly-once 보장

8. Latency vs Throughput

Latency

- [Throughput](#)
- [Spark vs Flink](#)
- 9. [Cluster에서 실제 실행 구조](#)
 - [Spark Cluster](#)
 - [Flink Cluster](#)
 - [핵심 차이](#)
- 10. [성능 튜닝](#)
 - [Spark 튜닝](#)
 - [Flink 튜닝](#)
- 11. [실제 현대 아키텍처](#)
 - [Batch 파이프라인](#)
 - [Streaming 파이프라인](#)
- 12. [Spark vs Flink 정리](#)

1. 왜 Spark / Flink가 필요한가

1.1 데이터 처리 패러다임의 변화

과거 데이터 시스템: 대부분 단일 서버 기반 처리 모델

전형적인 구조:

Application → Database → Script (Python / SQL) → CSV → 분석

다음과 같은 전제

- 데이터 크기가 제한적
- 데이터는 정적인 형태
- 분석은 오프라인에서 수행
- 데이터 생성 속도가 느림

하지만 현대 시스템에서는 이러한 전제가 무너지짐

1.2 현대 데이터 시스템의 특징

① 데이터 규모 증가

로그, 이벤트, 사용자 행동 데이터가 폭발적으로 증가

예:

- 하루 수십 TB 로그
- 초당 수만 ~ 수백만 이벤트

→ 단일 서버의 메모리와 CPU로는 처리 불가능한 규모

② 데이터는 지속적으로 생성된다

과거 데이터

Batch dataset

현대 데이터

Infinite event stream

예:

- 사용자 클릭
- 광고 노출
- IoT 센서
- 결제 이벤트

데이터는 끝없이 생성되는 스트림 형태가 됨

③ 실시간 요구 증가

과거 분석

데이터 수집 → 하루 뒤 분석

현재 요구

데이터 발생 → 몇 초 내 반영

예:

- 실시간 추천

- 실시간 이상 탐지
- 실시간 대시보드

④ 장애 필연적

분산 시스템에서는 다음이 항상 발생

- 노드 실패
- 네트워크 단절
- 디스크 오류

따라서 시스템은 장애를 가정하고 설계해야 한다

1.3 핵심 문제

현대 데이터 처리 시스템이 해결해야 하는 문제:

- 1 데이터가 너무 크다
- 2 데이터가 계속 들어온다
- 3 실시간 처리가 필요하다
- 4 장애가 항상 발생한다

1.4 필요한 시스템 특성

수평 확장 (Horizontal Scaling)

서버 하나가 아니라 "N개의 노드"로 계산을 분산한다.

병렬 처리 (Parallel Processing)

데이터를 여러 조각으로 나누어 동시에 계산한다.

예:

```
1TB 데이터
→ 100 노드
→ 각 노드 10GB 처리
```

장애 복구 (Fault Tolerance)

노드 하나가 실패해도,

전체 작업이 중단되지 않아야 한다.

분산 상태 관리 (Distributed State)

스트림 처리에서는 다음과 같은 상태가 필요하다.

예:

- 사용자별 클릭 수
- 세션 상태
- 실시간 집계

결론

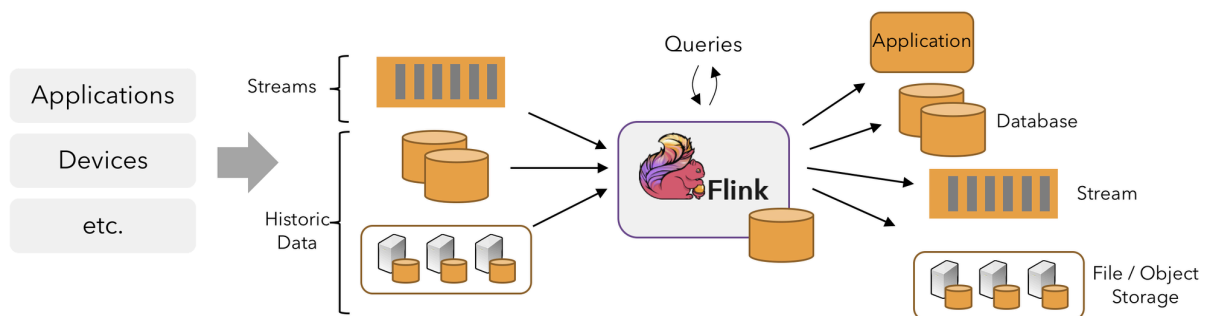
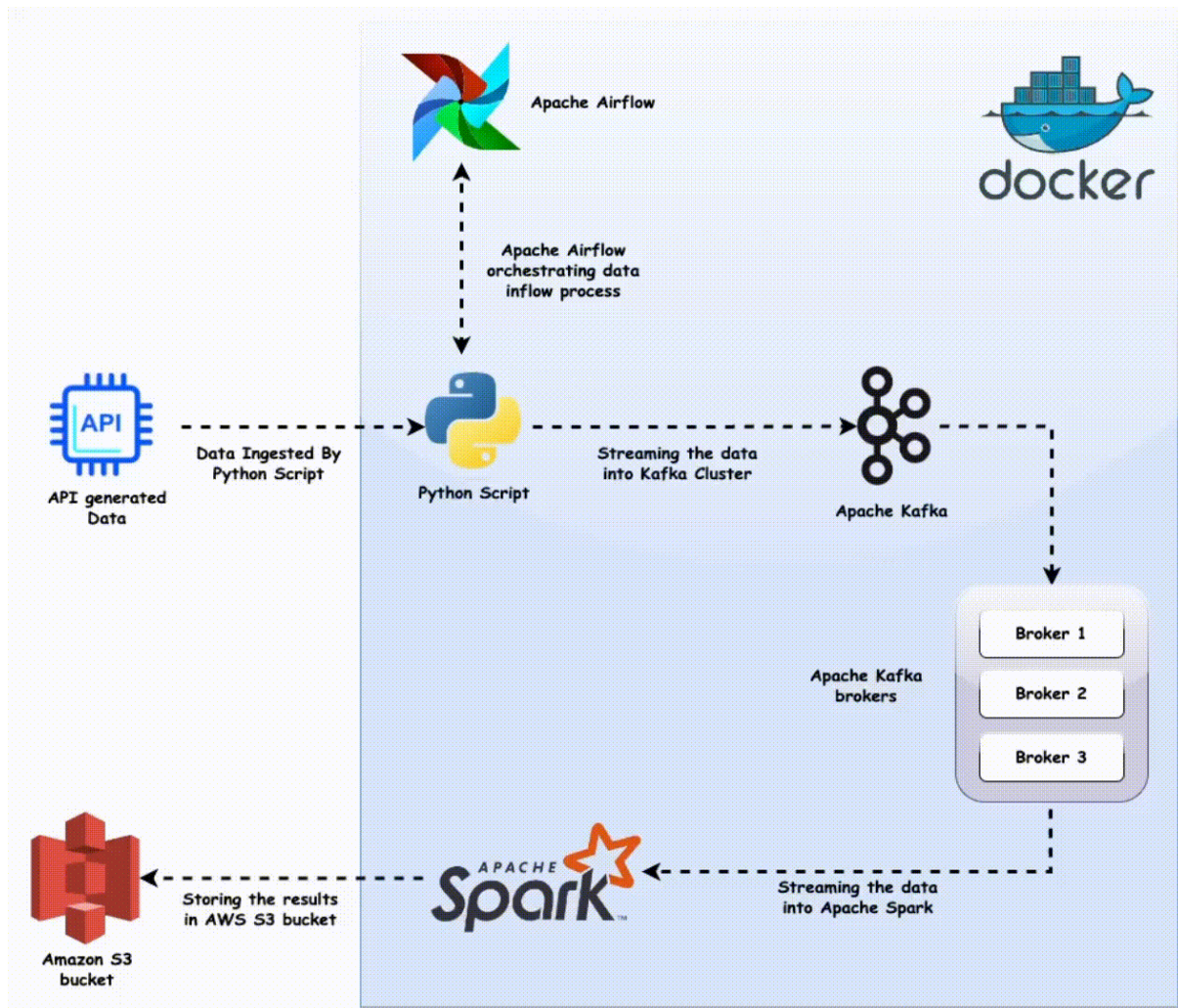
이 요구사항을 만족하기 위해 등장한 것

→ **Distributed Processing Engine**

대표적인 시스템

- **Apache Spark**
- **Apache Flink**

2. 데이터 파이프라인에서의 위치



현대 데이터 플랫폼은 일반적으로 다음 구조를 가진다.

```

Data Source
  ↓
Kafka / CDC / Log
  ↓
Spark / Flink
  ↓
Data Lake / Data Warehouse

```

각 계층의 역할

Source Layer

데이터 생성

예:

- Application logs
- Clickstream
- IoT events

Ingestion Layer

데이터 수집 및 버퍼링

대표 시스템:

- Kafka
- Pulsar
- Kinesis

역할:

- 데이터 스트림 저장
- 소비자 분리
- 백프레서 완화

Processing Layer

여기서 Spark / Flink가 등장

역할: 데이터를 계산하고 변환하는 계층

예:

- aggregation
- filtering

- feature generation
- enrichment

Storage Layer

처리된 데이터를 저장

예:

- S3
- HDFS
- Data Warehouse

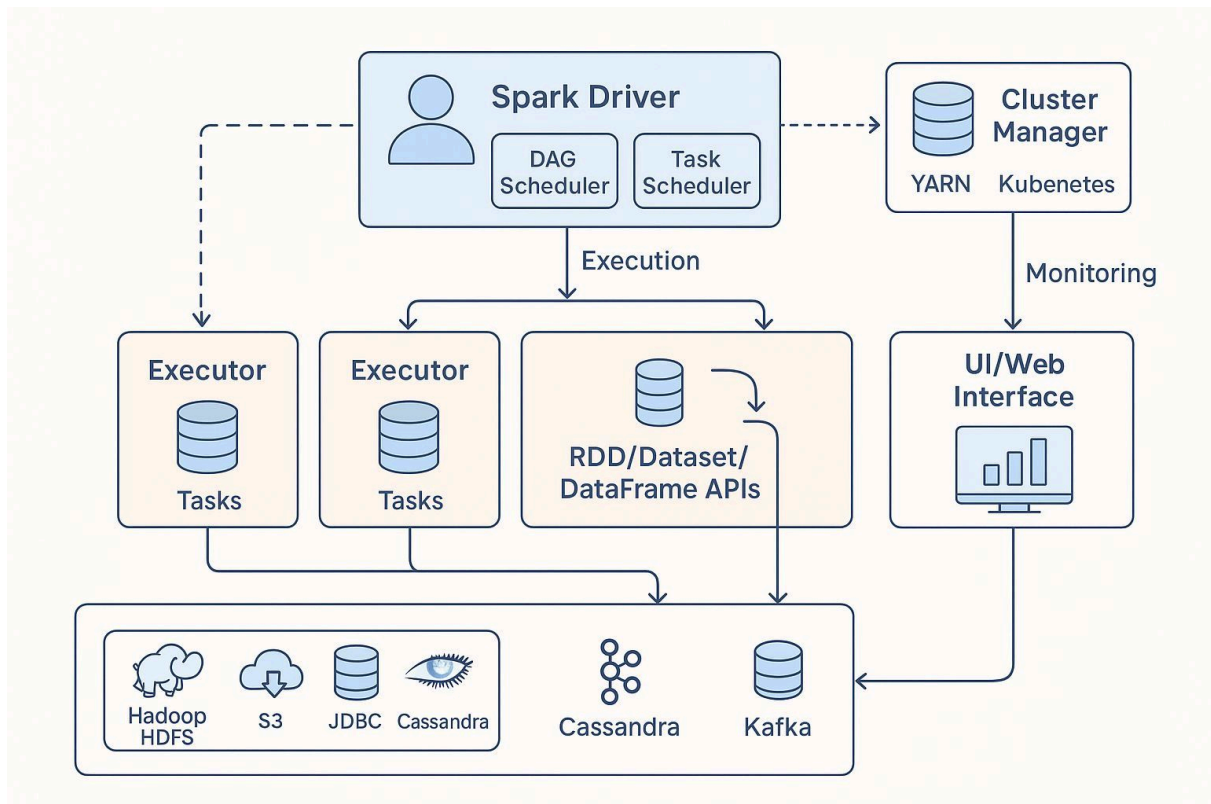
Serving Layer

분석 / 모델 / API에서 사용

예:

- BI Dashboard
- ML Training
- Recommendation

3. Spark 설계 철학



Spark의 핵심 질문

Spark의 출발점

: 엄청 큰 데이터를 어떻게 빠르게 계산할 것인가

Spark의 설계 철학

: **Batch-first distributed computation**

Spark의 기본 모델

DAG 기반 분산 계산 엔진

DAG (Directed Acyclic Graph)

데이터 처리 과정을 그래프로 표현한다.

예:

Read → Filter → Map → Join → Aggregate

Spark는 이 작업을 DAG로 구성한 후, 각 단계를 병렬 실행한다.

Spark Cluster 구조

Spark는 다음 구조로 동작한다.

```
Driver
↓
Executors
```

Driver

역할:

- 작업 DAG 생성
- 스케줄링
- Task 분배

→ Spark 애플리케이션의 컨트롤러

Executors

역할:

- 실제 계산 수행
- 데이터 처리
- 결과 반환

여러 노드에 분산되어 실행된다.

Spark 데이터 모델

RDD (Resilient Distributed Dataset)

Spark의 초기 데이터 모델.

특징:

- 불변 데이터
- 분산 저장
- lineage 기록

Lineage란 데이터를 어떻게 계산했는지 기록하는 그래프

Spark의 Fault Tolerance

Spark는 상태를 저장하지 않는다.

대신 **재계산 방식**을 사용한다.

예:

```
RDD1 → RDD2 → RDD3
```

노드 실패 시

RDD3을 다시 계산한다.

장점:

- 상태 관리 단순

단점:

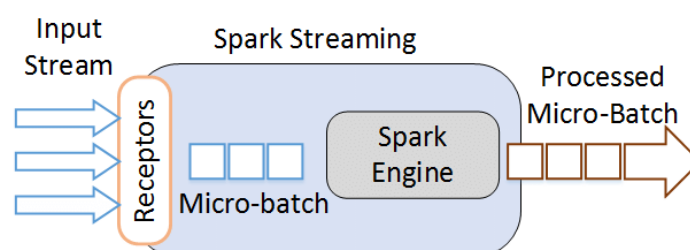
- latency 증가

4. Spark Streaming (Micro-batch)

Spark는 본질적으로

→ **Batch engine**

그래서 streaming 처리를 위해 **micro-batch 방식**을 사용한다.



Micro-batch 구조

데이터 흐름

```
stream events
↓
1초 단위 묶음
```

↓
batch processing

예:

[1초 데이터]
→ batch
→ 계산

장점

- 기존 Spark API 재사용
- 안정성 높음
- 구현 단순

단점

- latency 증가
- event ordering 문제
- late event 처리 어려움

5. Flink 설계 철학

Spark가 batch 중심이라면

Flink는 완전히 다른 철학을 가진다.

| 데이터는 원래 스트림 아닌가?

Streaming-first 모델

Flink에서는

Batch = bounded stream

즉, 모든 계산은

으로 수행된다.

6. Flink 실행 모델

Spark:

```
event → 모아서 → 계산
```

Flink:

```
event → 즉시 계산
```

Event Time 개념

스트림 처리에서 중요한 개념

: Event Time

예:

```
event 발생 : 10:00  
event 도착 : 10:05
```

Spark:

이미 batch 종료

→ 반영 불가

Flink:

event time 기준

→ 정상 반영 가능

7. Flink State + Checkpoint

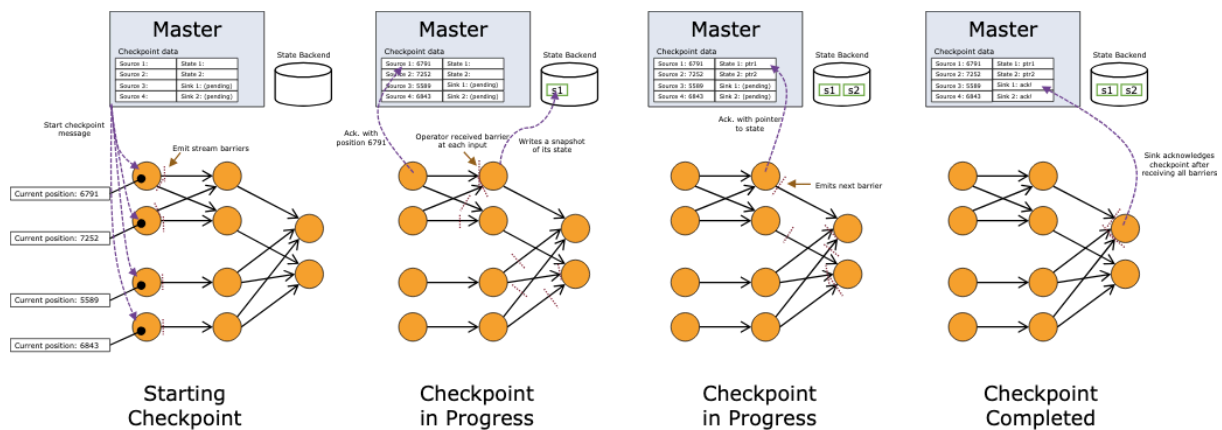
Flink의 핵심 기능

: Stateful Stream Processing

예:

- 사용자 클릭 수
- 세션 상태
- 실시간 집계

Checkpoint



Flink는 주기적으로 상태를 저장한다.

State
↓
Checkpoint
↓
Storage

노드 장애 발생 시

checkpoint restore

로 복구한다.

Exactly-once 보장

이 구조 덕분에 Flink는

Exactly-once processing

을 제공한다.

이는 실시간 시스템에서 매우 중요하다.

예:

- 결제 처리
- 금융 이벤트

8. Latency vs Throughput

Latency

입력 → 결과까지 걸리는 시간

Throughput

단위 시간당 처리량

Spark vs Flink

	Spark	Flink
처리 방식	Batch	Stream
Latency	초~분	ms
Throughput	매우 높음	높음
상태 처리	약함	매우 강함

9. Cluster에서 실제 실행 구조

Spark Cluster

구성

```
Driver
Executors
Resource Manager
```

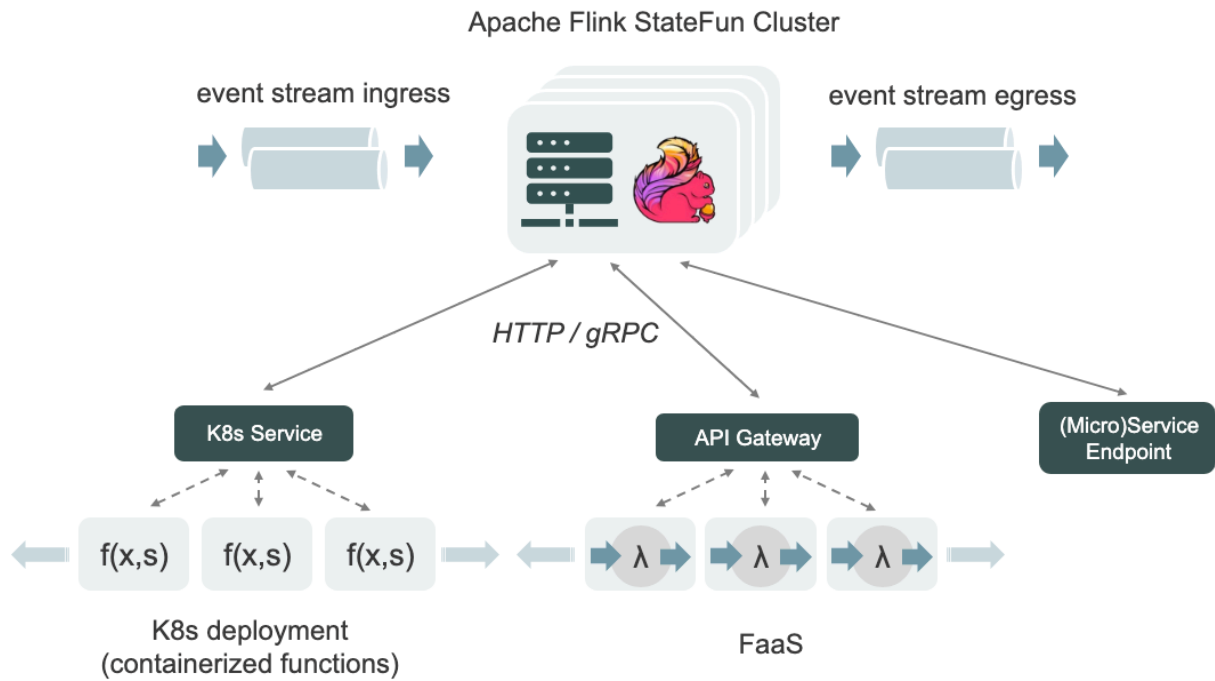
동작

1. Driver DAG 생성
2. Task 분해

3. Executors 실행

4. 결과 반환

Flink Cluster



구성

JobManager
TaskManager

동작

1. Stream graph 생성
2. Operator chain 구성
3. Slot 기반 실행
4. 지속적 처리

핵심 차이

Spark

Job 실행 → 종료

Flink

Job 실행 → 계속 실행

10. 성능 튜닝

Spark 튜닝

주요 요소

- executor memory
- executor cores
- shuffle partitions
- broadcast join
- caching

핵심 문제:

shuffle 비용 최소화

Flink 튜닝

주요 요소

- parallelism
- state backend
- checkpoint interval
- buffer timeout

핵심 문제:

state 관리

11. 실제 현대 아키텍처

현대 데이터 플랫폼에서는
Spark와 Flink가 **함께** 사용된다.

Batch 파이프라인

```
Kafka
↓
Spark
↓
S3 / Data Lake
↓
ML / Analytics
```

Streaming 파이프라인

```
Kafka
↓
Flink
↓
Realtime DB / Kafka
↓
Serving / Dashboard
```

12. Spark vs Flink 정리

	Spark	Flink
중심 철학	Batch-first	Streaming-first
Latency	초~분	ms
Throughput	매우 높음	높음
상태 처리	제한적	강력
운영 난이도	비교적 쉬움	복잡
사용 분야	ETL / ML	실시간 분석